THE UNIVERSITY OF SYDNEY

Due: Friday, Week 11 by 23:59

COMP3221 Assignment 2: Blockchain

Contents

1 Introduction

In this assignment, you will implement a peer-to-peer (P2P) blockchain system. A blockchain is a chain of blocks (like pages in a ledger), distributed across multiple nodes. Each block contains a set of transactions, which represent changes in ownership of digital assets or other logged information. By completing this project, you will create a network of blockchain nodes that communicate and cooperate to maintain a consistent distributed ledger.

1.1 Learning Objectives

- Understand key blockchain concepts and terminology by implementing them in code.
- Learn how a P2P network model operates and how to design communication between distributed nodes.
- Gain experience in developing a concurrent networked application (with sockets and threads).
- Apply principles of crash fault-tolerant consensus in a practical setting.
- Improve programming skills in networking and distributed system design, with a focus on blockchain mechanics.

2 Program Structure and Implementation Requirements

You may implement your solution in any programming language of your choice. However, your submission must include a wrapper script named `Run.sh` that serves as the single entry point. For example, if you choose Python, your wrapper script might look as follows:

```
1 #!/bin/bash
2 python3 your_solution.py "$@"
```

You are *not* permitted to use any libraries outside the core libraries provided for your programming language. For example, if you were to use Python, you may use libraries such as `socket`, but not `networkx`. Using any disallowed libraries will result in an immediate 0 for this assignment. If you are unsure if a specific library is allowed, please ask on Ed.

You are also permitted to use a build script that compiles your program. This script must be called `Build.sh`, and will execute before any of the test cases are run on your program.

You are permitted to use any libraries that come built into your chosen language, as well as a select range of extra libraries. For the list of allowed libraries, please see Appendix A.

3 Assignment Task

You will implement the software for a single **blockchain node** (peer). Running multiple instances of your node program will form a P2P network. Each node must act both as a **server** (listening for incoming requests from peers) and as a **client** (initiating requests to other peers). Nodes communicate by exchanging structured network messages over TCP connections. The ultimate goal is for all non-faulty nodes in the network to **reach consensus on a sequence of blocks**, thereby maintaining a consistent blockchain.

Every node will maintain a TCP server on a specified port to accept peer connections. It will also initiate outgoing connections to all other known peer nodes. The network topology is

fully connected: each node should have an independent, long-lived TCP connection to every other node in the peer list. Nodes must handle connectivity issues gracefully – for example, if a connection drops or a peer crashes, the node should detect this (via timeouts) and continue operating with the remaining peers.

Each node operates concurrently with multiple threads, for example:

- **Server Thread(s):** Accept incoming TCP connections from peers. For each connected peer, use a dedicated handler (thread) to receive and process messages from that peer.
- **Consensus (Pipeline) Thread:** Continuously monitor the node's transaction pool and coordinate the block proposal and consensus process (detailed below in Section 8).

All nodes in the network will run the same consensus protocol in synchronous rounds to agree on the next block to append. By the end of each consensus round, every correct node should have decided on the **same new block** to add to its blockchain (assuming not too many nodes have crashed).

3.1 Transaction Pool and Validation

Incoming transactions (from clients or other nodes) are collected in a **transaction pool** (mempool). The node will store transactions in this pool temporarily until they are included in a new block via the consensus process. Each transaction is a JSON object containing specific fields (see Section 5.2). Upon receiving a transaction (via a peer message), the node must **validate** it before adding it to its pool. **Invalid transactions must be explicitly refused**. The validation rules are:

- **Correct Nonce Sequence:** Each transaction has a `nonce` field, which must equal the number of transactions from the same sender that have already been **confirmed on the blockchain** (i.e. the sender's transaction count in the current chain state). For example, if a sender has 5 transactions in the blockchain (nonce 0 through 4), the next transaction's nonce must be 5. If a received transaction's nonce is not exactly one greater than the sender's confirmed nonce (or 0 if the sender has none yet), then the transaction is out-of-order or duplicate and **must be rejected**. There must be no two different transactions with the same sender and same nonce in the pool. (The nonce is monotonically increasing per sender, starting from 0.)
- **Signature Verification:** Every transaction is signed by its sender. The `signature` field must be a valid Ed25519 signature over the transaction's contents, and it must correspond to the given `sender` public key. The node must verify the signature using the sender's public key. If the signature is invalid, the transaction is **invalid** and should be rejected.
- **Field Formats:** All transaction fields must adhere to the specified format. If any field is malformed (e.g. wrong length or containing non-alphanumeric characters), the transaction is **invalid**.

When a transaction is rejected due to any of the above rules, the node should **not include** it in its pool or any future block. Only transactions that pass all validation steps are pooled.

3.2 Block Creation and Addition

Periodically, the node will create a new block containing a batch of pending transactions from its pool and attempt to append this block to the blockchain via the consensus protocol. A node **proposes** a new block when its transaction pool becomes non-empty (triggering an attempt to form the next block).

Each block is a JSON object with the following keys (see Section 5.2 for full schema):

- `index`: The block's position in the chain (1 for the genesis block, 2 for the first block after genesis, etc.).
- `transactions`: A list of transaction objects, each containing the fields `sender`, `message`, `nonce` and `signature`.
- `previous_hash`: The `current_hash` of the previous block in the chain (for the genesis block, a predefined value is used – see Appendix B).
- `current_hash`: The SHA-256 hash of **this block's content** (computed over the string representation of the block's other fields, with keys sorted lexicographically for consistency).

When a node creates a block proposal, it must set the `index` to (current blockchain length + 1), include a selection of transactions from its pool, and set `previous_hash` to the hash of its latest block. It then computes the `current_hash` for the new block. This block will then enter the consensus protocol to decide whether it will be the next block appended to the chain.

After a consensus round (see Section 8), the nodes will have agreed upon one block to commit. When a node **decides** on a block for a given round, it appends that block to its local blockchain. The node should then update its state accordingly:

- Execute/confirm the block's transactions (e.g. increment nonce counters for senders).
- Remove the block's transactions from the transaction pool (since they are now confirmed on-chain).
- If any other pending transactions in the pool became invalid due to this new block, those should be removed as well. For example, if multiple transactions from the same sender were in the pool and one of them was included in the block, any other transaction from that sender with the same nonce or an out-of-order nonce should be dropped.

The blockchain should always remain valid: a decided block's `previous_hash` must match the hash of the previously decided block. All nodes start with the same genesis block, so if the consensus protocol works correctly, every node's chain will evolve identically over time (aside from any blocks that crashed nodes failed to append).

4 Network Protocols and Message Formats

All network communication between nodes uses a simple message protocol over TCP. Nodes exchange messages that are length-prefixed and encoded in JSON. Each message sent over the network is preceded by a 2-byte integer length field (unsigned, big-endian byte order) that specifies the number of bytes in the message body. The message body immediately follows the length field and consists of a JSON-encoded object. The maximum message length is 65535 bytes (0xFFFF), as the length is 16-bit. This framing allows the receiver to know how many bytes to read for each message from the continuous TCP stream.

The message body is a JSON object with two top-level keys: `"type"` and `"payload"`. The `"type"` field is a string indicating the kind of message, and `"payload"` contains the data relevant to that message. There are two types of messages in this protocol:

- `"transaction"` — used to submit a transaction to a node.
- `"values"` — used during consensus to exchange proposed block values (hence "values" refers to proposed blocks).

4.1 Transaction Message Format

A message with `"type": "transaction"` carries a single transaction to be processed. The payload in this case is a JSON object representing the transaction. The transaction JSON has the following keys:

- **`sender`** (string): The hex-encoded 32-byte Ed25519 *public key* of the sender (i.e. the account initiating the transaction). This is a 64-character lowercase hexadecimal string. This public key uniquely identifies the user/account.
- **`message`** (string): Arbitrary UTF-8 text authored by the sender. The consensus protocol treats this as opaque data. The message contains up to 70 alphanumeric characters and spaces.
- **`nonce`** (integer): A number indicating how many transactions the `sender` has already had confirmed in the blockchain before this one. The nonce starts at 0 for each sender's first transaction, and increments by 1 with each confirmed transaction. This field is used to ensure each transaction is unique, and for a given sender (preventing replay attacks or double-spending).
- **`signature`** (string): The hex-encoded 64-byte Ed25519 *signature* of the transaction. This signature is generated by the sender's *private key* over the contents of the transaction (the transaction fields `sender`, `message` and `nonce` concatenated in this order). It is 128 characters in hexadecimal. The signature proves the authenticity of the transaction (that it was indeed created by the holder of the private key corresponding to `sender`).

When a node receives a `"transaction"` message, it should parse the payload as a transaction dictionary and run the validation rules (nonce check, signature verification, etc.) as described in Section 2.1. After processing, the node should return a response to the sender (outside the scope of this message format, a simple `True` or `False` as mentioned). The original message does not itself demand a response in JSON format, but the sending function will expect a boolean acknowledgement.

Example — Transaction Message: Suppose a user with public key `"a578...5363b"` wants

to log the message `"Never Gonna Give You Up"` with nonce `0`. The user signs the transaction, producing signature `"142e...60c7fd0c"`. The network message sent to the node would have the following JSON body (length prefix omitted here for clarity of the output):

```
{
  "type": "transaction",
  "payload": {
    "sender": "
   a57819938feb51bb3f923496c9dacde3e9f667b214a0fb1653b6bfc0f185363b",
    "message": "Never Gonna Give You Up",
    "nonce": 0,
    "signature": "142e395895e0bf4e4a3a7c3aabf2..... (and so on)"
  }
}
```

The first two bytes of the actual transmission would be the length of this JSON string in bytes (in big-endian). The node, upon receiving these bytes, will reconstruct the JSON and interpret it accordingly.

The node's response to this message (over the same TCP connection) should be a standalone JSON value `true` (if accepted) or `false` (if rejected), or simply the literal `True/False` as a Python boolean serialised.

4.2 Block (Values) Message Format

During the consensus protocol, nodes exchange messages of `"type": "values"`. These messages carry block proposals (or requests for them). The `"payload"` for a `"values"` message is a JSON structure that can vary depending on context:

- In our implementation, every `"values"` message carries the sender's own current block proposal (a list containing its block, or an empty list if it has none yet). Sending your proposal thus implicitly requests that the peer reply with its own `"values"` message containing that peer's proposal.
- When **responding** or broadcasting values, the payload is a **list of block objects**. Each block object in the list is a JSON dictionary with the fields described above (`index`, `transactions`, `previous_hash`, `current_hash`). In our consensus algorithm (a single round to decide one block), each node will have exactly one block proposal per round. Thus, you may often send or receive a `"values"` message where the payload list contains either one block (the peer's proposal) or multiple blocks (if a node bundles all proposals it knows). In a fully synchronous broadcast approach, a node might send its own proposal to everyone (payload list of one), and also later send a list of all proposals it received (payload of many) — but our simplified approach mainly uses one exchange of single blocks.

Every block in the payload list should be formatted as valid JSON with its keys and values as described. For completeness, here's a breakdown of a block object (these apply whether the block is sent in a values message or printed to stdout):

- `index`: (integer) Block's position in chain.
- `transactions`: (array) List of transaction objects included in the block.

- `previous_hash`: (string) The hash of the previous block in hex.
- `current_hash`: (string) The hash of this block in hex.

Example — Values Request: Node A wants to request Node B's proposed block for the current round. Node A sends a message to B:

```
{ "type":"values", "payload":[ Block_A ] }
```

and receives the following response:

```
{ "type":"values", "payload":[ Block_B ] }
```

This indicates a request. When a node receives a values request (empty list), it must respond with a values message whose payload is its own current proposal (or [] if it has none).

Example — Values Response (Single Block): Node B has a block proposal (say Block_B) ready. It responds to A with:

```
{
  "type": "values",
  "payload": [
    {
      "index": 2,
      "transactions": [
        {
          "sender": "b1c22f...5d8e",
          "message": "Touch grass",
          "nonce": 0,
          "signature": "8f23ab...45d1"
        }
      ],
      "previous_hash": "<hash_of_block_1>",
      "current_hash": "<hash_of_block_B>"
    }
  ]
}
```

Here, the payload list contains one block object (Node B's proposal for block #2).

Example — Values Broadcast (Multiple Blocks): If Node A, after collecting proposals from B (and maybe others), decides to broadcast the full set, it might send:

```
{
  "type": "values",
  "payload": [
    { ... Block_A ... },
    { ... Block_B ... },
    { ... Block_C ... }
  ]
}
```

with each block from a different node. In all messages, JSON syntax must be strictly followed. That means:

- Keys and string values in quotes `" "`.
- Lists `[...]` and objects `{...}` with commas between elements.
- No trailing commas.
- True/False in JSON should be lowercase `true/false` if they appear.
- Numeric values (nonce, index) appear as numbers (no quotes).

In summary, design your networking code such that:

- Before sending, you prepend the 2-byte length.
- You then send the JSON text of the message.
- When receiving, you first read 2 bytes to get N, then read N bytes to get the JSON message.

5 I/O Test Case Rules

Your program will be tested with automated scripts that send specific inputs (transactions, network messages, etc.) and check your program's **stdout output** and behaviour. It is crucial that your program's output **matches exactly** the expected format for the test cases. Any deviation (extra messages, different formatting, etc.) can cause the autograder to mark your solution as incorrect, so adhere strictly to the format described here.

5.1 Standard Output Logging Format

5.1.1 Valid Transactions

When your node receives a new **valid transaction** (either from a client or a peer) and adds it to the pool, it should immediately print the transaction JSON object to stdout. The transaction should be printed exactly as a JSON object with its fields. For example, a valid transaction might be logged as:

```
{
  "type": "transaction",
  "payload": {
    "sender": "
   a57819938feb51bb3f923496c9dacde3e9f667b214a0fb1653b6bfc0f185363b",
    "message": "Never Gonna Give You Up",
    "nonce": 0,
    "signature": "142e395895e0bf4e4a3aabf2f59c3abc45c9c2b..."
  }
}
```

This log entry corresponds to the node receiving a transaction (with sender's public key, message, nonce 0, and the signature as shown) that passed validation and was accepted. **Important:** The JSON should use double quotes for keys and string values, and the formatting (spaces/newlines) should be consistent. You must pretty-print the JSON exactly with 2 spaces for indentation and with keys in lexicographical order (e.g. using Python's `json.dumps(obj, sort_keys=True, indent=2)`). Any deviation in whitespace or key ordering will result in a mismatch against the expected output. Using a JSON library to dump the object can ensure correctness. The output **must exactly** match the expected JSON format down to every character.

5.1.2 Rejected Transactions

If an **invalid transaction** is received (wrong format, incorrect nonce, bad signature, etc.), it should **not** be printed in the above manner. In fact, by default, you should not print anything for a rejected transaction (since it did not enter the pool). The only feedback for a rejected transaction should be the network response (`False`) back to the sender. Invalid transactions must be rejected according to each validation rule (malformed public key, bad signature, incorrect nonce, pool-full, etc.). Your node must respond with `False` for each such case and must not add the transaction to its pool.

5.1.3 Blocks (Consensus Results):

When a consensus round completes and a new **block is decided** to be appended to the blockchain, each correct node should print the full block JSON object to stdout. This should happen exactly once per round, at the moment the block is appended. The block should be printed in JSON format, similar to transactions but with the block fields. For example, a block might be logged as:

```
{
  "index": 2,
  "transactions": [
    {
      "sender": "
   a57819938feb51bb3f923496c9dacde3e9f667b214a0fb1653b6bfc0f185363b",
      "message": "Never Gonna Give You Up",
      "nonce": 0,
      "signature": "142e395895e0bf4e4a3a7c3aabf2f.... (and so on)"
    }
  ],
  "previous_hash": "<hash_of_block_1>",
  "current_hash": "<hash_of_this_block_2>"
}
```

This indicates the node has appended block #2. The `transactions` list contains one transaction (the same one printed earlier with nonce 0). The `previous_hash` should exactly match the `current_hash` of the prior block in your chain (block #1, the genesis block in this case), and `current_hash` is the SHA-256 of this block's content. The exact values of the hashes will depend on your implementation and the genesis block; in the example above, they are shown as placeholders. Your program should compute and fill these in correctly. **Formatting:** As with transactions, the block JSON should be pretty-printed or consistently formatted. Ensure the JSON syntax is correct and keys are quoted. You should sort the keys lexicographically in the printed output (this is required for computing `current_hash` and also makes outputs comparable).

5.1.4 Consensus Progress (Recommendation)

The primary required outputs are the transactions and decided blocks as described. In addition, you might want to log certain consensus steps (like sending/receiving block proposals or the final decision) when testing your program locally. Be cautious: any additional output must match expected values if the autograder checks for them. The autograder will only check for the final block outputs (and transaction outputs) in the correct order. It will not check intermediate text like "consensus started" unless we specify it. For this assignment, **you are not required to output intermediate consensus messages** to stdout. In most cases, the autograder will be able to identify these logs and will omit them from the expected output, but do not rely on this; it is recommended that you remove any print statements from your program that do not directly satisfy the testing requirements. A dedicated testing environment will also be provided for you to run your own tests.

5.2 JSON Specifications

All outputs on stdout will be compared exactly against the expected logs. That means your JSON spacing, commas, braces, and newlines must be exactly as in the examples. For instance, if the expected output prints each key on a new line with 2 spaces of indentation, your output should do the same. Conversely, if the expected output is all on one line, you should match that. Pay attention to any examples provided in the assignment or test cases. Using Python's `json.dumps(obj, sort_keys=True, indent=2)` (or equivalent in other languages) is a good way to produce stable, correctly formatted JSON strings. Do not print Python-specific representations (e.g. using single quotes for keys or booleans as `True/False` capitalised) – these will not match JSON exactly. Always output JSON booleans as `true` or `false` (lowercase) if included in an object.

Below are some example scenarios:

Table 1: Examples for Transactions and Consensus Rounds

Example Name	**Input**	**Node Action**	**Output**	**STDOUT**
Single valid transaction	A client submits a transaction with nonce 0, proper signature, etc.	The transaction is valid and added to the pool.	The node prints the transaction JSON (pretty-formatted) as shown in the transaction example. It creates block #2 with that transaction, reaches consensus and then prints the new block JSON.	The transaction JSON is printed first; later, the block JSON is printed.
Transaction with incorrect nonce	A client/peer submits a transaction from sender X with nonce 5, but sender X has no prior transactions (expected nonce 0).	Transaction is recognised as invalid (nonce out of sequence) and is rejected—thus, not added to the pool.	No transaction JSON is printed; a network response of `False` is sent (but not shown on stdout).	No output on stdout.
Transaction with invalid signature or format	A transaction arrives with a malformed signature (invalid Ed25519 signature for the given sender).	Signature verification fails, so the transaction is rejected.	No transaction JSON is printed.	No output on stdout.

6 Execution Guidelines

Your submission must include a shell script called `Run.sh` at the root. This script is used to start your node program. The testing system will invoke your program through this script. The usage must be:

```
./Run.sh <Port-Server> <Node-List-File>
```

Here, **<Port-Server>** is the port on which the node will run, and **<Node-List-File>** is a filename (ending in `.txt`) where each line is in the following format:

```
host:port
```

Your node loads this at startup to know each peer's TCP address.

For example, if you name it `nodes.txt`, its contents might be:

```
127.0.0.1:5001
127.0.0.1:5002
```

Clients will submit pre-signed transactions; the node program itself only validates transaction signatures and does not create or sign any messages.

Suppose we want to run a network of three nodes on the same machine for testing. We might use:

Node1: `./Run.sh 5000 Node1_Peers.txt`
Node2: `./Run.sh 5001 Node2_Peers.txt`
Node3: `./Run.sh 5002 Node3_Peers.txt`

This ensures each node knows the others' addresses. The script should launch your program such that Node1 listens on 5000 and connects to 5001 and 5002, etc.

Ensure that threads are properly synchronised where necessary (for example, access to the transaction pool or blockchain should be thread-safe). The consensus algorithm as specified is a **synchronous round-based protocol**; however, your implementation will rely on timeouts to detect failures. The timing requirements are as follows:

- **Startup:** When establishing connections to peers at startup, do not set an immediate timeout. It's possible some peers aren't up at the exact same time. Your node should attempt to connect without a timeout initially (blocking connect or a reasonably long timeout) to allow all nodes to come online. Once the connection is established, keep it open.
- **Consensus Round Communication:** When sending requests to peers for their block proposals (the "values request"), you **must set a timeout of 2 seconds** on those socket operations. This means after sending a request, your node will wait up to 2 seconds for a response from that peer. If 2 seconds pass without a response, assume that the peer is unresponsive (possibly crashed) for this round. In this case, you should **mark that peer as crashed** and stop waiting for it in the current and future consensus rounds.

- **Post-Round Reset:** After a consensus round completes, for any peers that did respond and are still considered alive, you should remove the timeout (or set it back to an indefinite/blocking mode) for the connection during normal operation. This allows the connection to be used to receive transactions or other messages at any time without inadvertently closing due to a timeout. Essentially, use timeouts *only* during the specific phase of requesting block values, not for the entire duration of the connection.

Make sure to implement the socket timeout using functions like `socket.settimeout(5)` in Python (and then `settimeout(None)` to disable after).

Your program should handle unexpected events gracefully. For example, if a peer disconnects suddenly (socket read returns EOF) and treat the peer as crashed (don't spam output, just mark internally). The node should continue running with the remaining peers. If the network splits or too many nodes crash (more than f), the consensus might not succeed; in such cases, it's acceptable that the round fails, but the node should not deadlock or crash itself.

Ensure that your threads terminate when the program is instructed to end (for instance, in our tests, we might call `stop()` or simply kill the process after some time). Daemon threads or proper join on threads can be used. Sockets should be closed on exit to free the ports quickly (especially important in automated testing where multiple runs happen).

7 Submission Instructions

Deadline: The final version of your assignment must be submitted electronically via Ed **by 23:59 on Friday of Week 11**. Plan to submit well before the deadline – last-minute issues will not excuse late submissions. Late submissions incur a penalty of 5% of the assignment grade per day, up to a maximum of 10 days late (after which submissions may not be accepted). As a reminder, **simple extensions are automatically applied for this unit**.

Submission Contents: You are required to submit **your source code and a brief** README.md **file.** Specifically:

- **Source Code: Submit all your source code files individually.** This means all `.py` files (or files in the language you used) that make up your node program, plus the `Run.sh` script and any other necessary scripts.
- **Compilation/Execution Instructions:** How to compile (if needed) and run your program. For Python, this might just be "requires Python 3.x, run the provided `Run.sh` script".

Reminder

Your program needs to run correctly in the testing environment. Your program running correctly on your own machine and system architecture will not be considered grounds for alternative marking; as a result, you are encouraged to continuously make submissions to Ed and validate your program against the test cases on a rolling basis, and not wait until close to the deadline to verify that your program passes all of the checks.

8 The Consensus Protocol

This assignment uses a **synchronous crash fault-tolerant consensus algorithm** to ensure all nodes agree on the next block to add, even if some nodes crash or some messages are lost. We outline here the specific protocol each node should follow in each consensus round (each round produces one block for the chain). Note that this follows the same protocol defined in the tutorials, which you are welcome to use freely for this assignment.

First, some definitions:

- Let n be the total number of nodes in the network (peers participating in the consensus).
- We define $f = \lceil n/2 \rceil - 1$. This f is the maximum number of nodes that may crash (fail silently) such that the remaining nodes can still reach consensus. In other words, the protocol can tolerate up to f crash failures. This formula effectively means more than half the nodes must remain operational. For example, if $n = 5$, then $f = \lceil 2.5 \rceil - 1 = 3 - 1 = 2$ (at most 2 nodes can crash). If $n = 4$, $f = \lceil 2 \rceil - 1 = 2 - 1 = 1$ (at most 1 crash). If $n = 3$, $f = \lceil 1.5 \rceil - 1 = 2 - 1 = 1$ (at most 1 crash). The value of f is fixed and must be used throughout your implementation; inconsistent handling of timeouts or fault marking may result in non-deterministic behaviour.
- **Round structure:** Time is conceptually divided into rounds (although in implementation, we trigger rounds based on events rather than a global clock). Each round aims to decide one block. In a healthy network with frequent transactions, rounds will happen back-to-back as new blocks are proposed continuously. If there are no new transactions, no round is needed until one arrives (the node will wait idle).

Each node's consensus (pipeline) thread should perform the following high-level loop for each round:

1. **Triggering a New Round – Wait for Transaction Pool Ready:** The node does not start a consensus round until it has something to propose. Two events can trigger this:
   - The node's own transaction pool becomes non-empty (e.g. a client submitted a transaction and it passed validation). This means the node has a value (block) it can propose.
   - The node receives a **consensus request** from another node (a peer sends a `"values"` request message). This indicates another node is starting a round (because that node got a transaction and is ready to propose a block). Even if the current node's pool is empty, it must participate in the round when asked — which means it should propose an "empty block" (a block with no transactions) if it has nothing, so that it can still take part in consensus.
2. **Block Proposal Creation:** Once the node decides to start the round, it gathers all current transactions from its pool and creates a proposal:
   - `index` = (last block's index + 1).
   - `transactions` = list of transactions collected.
   - `previous_hash` = hash of last block in the chain (for genesis, see Appendix B).
   - Compute `current_hash` for this block (SHA-256 of the block's content with sorted keys).

This block is now the node's proposal for the round. Even if the transactions list is empty, the node can still form a block with the correct index and previous_hash.

3. **Exchange Proposals (Broadcast Phase):** Each node must learn about other nodes' proposals. This is achieved via a **unified request/response exchange with each peer**:
   - For each peer in the node's peer list, the node initiates a TCP request for that peer's block:
     - Send a `"values"` message whose payload is your block proposal (or `[]` if you have none). This implicitly requests the peer's own proposal.
     - Wait exactly 2 seconds for the peer to respond with its `"values"` message containing the block. If the peer responds in time, parse the block from the message and record it in your set of proposals.
     - If no response is received within 2 seconds, mark that peer as **crashed** and do not include its proposal in this round.
   - Concurrently, upon receiving any `"values"` message, immediately respond with your own `"values"` message containing your current block proposal.

   By the end of this phase, each node has:
   - Its own proposed block.
   - Block proposals from the non-crashed peers.

4. **Decision on Block (Consensus Decision):** Once a node has collected all available proposals (or marked some peers as crashed), it decides on the block for this round:
   - If **at least one proposed block contains at least one transaction**, then ignore any empty block proposals.
   - Among the remaining blocks, choose the block whose `current_hash` is lexicographically smallest (treat the hash as a hexadecimal string).
   - In the case that all proposals are empty, choose the one with the lexicographically smallest `current_hash`.

   This ensures that all correct nodes choose the same block (denote it as *B_decided*).

5. **Commit the Decided Block:** The node appends *B_decided* to its blockchain:
   - Add the block to the blockchain structure (increment chain length).
   - Update internal state (mark transactions in that block as confirmed, update sender nonces).
   - Remove conflicting or now-invalid transactions from the pool.

6. **Prepare for Next Round:** The node returns to waiting for a new trigger (new transactions or a peer request). Note that if some nodes crash, they remain marked as such.

Crash Fault Tolerance: The protocol must tolerate up to f crashes (where $f = \lceil n/2 \rceil - 1$). As long as more than $n/2$ nodes are operational, all correct nodes will append the same block. Timeouts ensure that if a node is unresponsive, the protocol proceeds without indefinite waiting.

In our tutorials, a simpler consensus choosing the minimum of integers was implemented. Here, blocks are compared based on their computed hash (with the added rule of ignoring

empty blocks when non-empty ones exist). You may use your code from that lab (as well as the solutions) freely without reference.

9 Test Cases and Marking

Test cases are divided into:

- **Public Test Cases:** Both the input and output of each case are visible. You may wish to find the input for a test case by printing any information passed to your program.
- **Hidden Test Cases:** The input and output of these test cases are hidden, but you can still see whether or not they pass.
- **Private Test Cases:** The input and output are hidden, and you will not be able to see if you pass these test cases until after the deadline.

Each test case carries a specific weight, and your overall mark is determined by the cumulative weighted score across all test cases. Your program will be tested only against the criteria specified in this assignment.

Academic Honesty and Plagiarism

By submitting your assignment via ED, you agree to abide by the University policies on academic honesty. All work must be your original work. If any part of your submission is not your own, you must immediately inform your tutor or lecturer. Refer to the policy slides from Week 1 for further details. The School of Computer Science may share your assignment with a plagiarism checking service.

From Semester 1 2025, you can use AI tools like Microsoft Copilot Chat for your assignments, unless your unit coordinator has prohibited the class from using it. For exams and tests, you will not be allowed to use AI unless your unit coordinator has expressly permitted it. Different units and assessments will have different rules.

At the same time, it's important to understand when the use of AI is unethical, inappropriate or breaches the University's rules about academic integrity. Submitting assessments that aren't your original work – including work produced by AI – may constitute a breach of academic integrity.

In assessable work, the unapproved use of automated writing tools or generative AI, or failing to acknowledge them, is currently considered to be a breach of academic integrity, and penalties may apply. If you're unsure about whether you can use AI or how you can use it, it's important that you speak with your unit coordinator.

For general learning purposes, as opposed to assessments, you are permitted to use generative AI tools, as long as you follow all University policies including the Academic Integrity Policy 2022, Acceptable Use of ICT Resources Policy 2019 and the Student Charter 2020.

A Allowed Libraries

Table 2: Allowed Libraries and Dependencies

Language	Allowed Libraries & Packages
Python	`PyNaCl`, `ed25519`
Java	Jackson (`com.fasterxml.jackson.core`, `.databind`)
Rust	`crossbeam`, `serde/serde_json`, `sha2`, `ed25519-dalek`
C	`jansson`, OpenSSL (`openssl/sha.h`) or `libsodium`
C++	`Boost.Asio`, `nlohmann/json`, OpenSSL or `libsodium`
C#	`Newtonsoft.Json`, `Nsec.Cryptography`
Go	`golang.org/x/crypto/ed25519`

B Node Discovery and Initialisation

Nodes are **pre-configured with a list of peers**. When starting a node, you provide it with the addresses of all other nodes (via the `<Node-List>` argument to the run script). The node uses this list to know which peers to connect to.

The `<Node-List-File>` is the path to a text file where each line is:

```
host:port
```

Your node reads this at startup to know each peer's TCP address. You may assume the file is well-formed.

No Dynamic Joins: All nodes start around the same time with the full list. You do not need to implement additional peer discovery.

Startup Timing: Some nodes may start before others. Therefore, a node should continuously attempt to connect until successful or be marked as crashed (if it never responds).

C Genesis Block and Initial State

Every node must start with a **genesis block** in its blockchain. The genesis block is block #1 (index = 1) and serves as the root of the chain.

It contains:

- `index`: The block's position in the chain (for genesis, set to 1).
- `transactions`: An empty list.
- `previous_hash`: A predefined constant (64 zeros).
- `current_hash`: SHA-256 hash of the block content (with sorted keys).

All accounts are expected to start with a nonce of 0.

To initialise the blockchain, you must:

- Create the genesis block (index=1, transactions=[], previous_hash="<constant>", compute current_hash).

- Initialise the blockchain structure with the genesis block.
- Set up internal state (e.g. nonce tracking for each sender).

The genesis block is not printed to stdout during normal operation.

D Reference Hashing Script

The script below shows exactly how this specification computes a block's `current_hash`. It *must* be run on the JSON text produced by `json.dumps(obj, sort_keys=True, indent=2, separators=(',', ': '))` so that all nodes derive identical digests. You may invoke this program from your implementation if you wish, but remember to remove the print statements.

```
"""
Usage:
    ./hash_json.py '<json-string>'
"""
import json
import sys
import hashlib

def hash_json():
    if len(sys.argv) != 2:
        sys.exit("Usage: hash_json.py '<json-string>'")

    raw = sys.argv[1]

    try:
        obj = json.loads(raw)
    except json.JSONDecodeError as err:
        sys.exit(f"Invalid JSON: {err}")

    canonical = json.dumps(
        obj,
        sort_keys=True,
        indent=2,
        separators=(',', ': ')
    )

    digest = hashlib.sha256(canonical.encode('utf-8')).hexdigest()

    print(canonical)
    print("\nSHA256:", digest)

    return (canonical, digest)
```